

自動化と外部連携で学ぶ 用語まとめ(第 6 章)

～ デザインパターンの型から、フック自動化・MCP 連携・カスタムコマンドまで ～

この文書は、フックによる自動化と、MCP での外部サービス連携でよく出てくる言葉を、身近な例えとセットで分かるようにまとめたものです。

作成日:2026 年 6 月 22 日

はじめに — この文書の使い方

この文書は、用語の意味を初学者向けにやさしくまとめたものです。むずかしい言葉は、レストランや建物などの身近なものにたとえて説明しています。

各用語は「ひとことで言うと → 身近な例え → 具体例 → つながり」の同じ型で並べています。まずは「ひとことで言うと」だけ拾い読みしても大丈夫です。

◆ この文書で扱う用語(テーマ別)

- **自動化を支える設計の型:** デザインパターン / ファクトリクラス / 抽象ファクトリ / サービスクラス
- **フックで自動化する(基礎):** hooks / フック / matcher / PreToolUse / PostToolUse
- **フックの中身を書く:** Write|Edit / Bash / シェルスクリプト / stdin 経由 / 一次フィルタ
- **フックの実例と仕上げ:** pre-commit フック / ビルドコマンド / CI/CD パイプライン / カスタムスラッシュコマンド / Codex CLI
- **外部サービスとつなぐ(MCP):** MCP / ハブ / ブリッジ / Jira / PostgreSQL / Slack / Timetracker-notification

第 1 部 — 自動化を支える設計の型

1. デザインパターン(Design Pattern)

ひとことで言うと:よくある設計の悩みに対して、先人がまとめた「定番の作り方」の見本集のこと。

料理の「定番レシピ集」のようなものです。毎回ゼロから考えなくても、似た場面では実績のある作り方をなぞれば、読みやすく壊れにくいものが作れます。

◆ デザインパターンのうれしさ

- 同じ悩みを毎回考え直さずに済む
- 名前で会話でき、意図が伝わりやすい
- 実績のある形なので失敗しにくい

【つながり】 そんな定番レシピの中で、「物を作る手間」を一箇所に任せる代表格が、次の「ファクトリクラス」です。

2. ファクトリクラス(Factory Class)

ひとことで言うと:「何が欲しいか」を伝えると、対応する部品を作って渡してくれる製造窓口役のクラスのこと。

レストランの厨房に「これください」と頼むと、出来た品を出してくれる注文窓口のようなものです。使う側は中の作り方を知らなくてよく、欲しい物の名前を伝えるだけで受け取れます。

◆ ファクトリの呼び出し(イメージ)

```
// 「これ作って」と頼むと、出来た品を返してくれる
Button btn = WidgetFactory.create("button");
// → 使う側は中の作り方を知らなくてよい
```

【つながり】 窓口が1個ずつ作るのに対し、関連する部品を「ひとそろい」まとめて作る上級版が、次の「抽象ファクトリ」です。

3. 抽象ファクトリ (Abstract Factory)

ひとことと言うと:互いに関連する部品を、系統ごとに「ひとそろい」まとめて作り分ける仕組みのこと。

和食・洋食といった「コースを丸ごと一式そろえる工場長」のようなものです。和のコースを選べば器も箸もまとめて和で揃い、洋を選べば一式が洋で揃う、と組み合わせの取り違えを防ぎます。

◆ 系統ごとに一式そろえる(イメージ)

```
// 「和食」工場を選ぶと、器も箸も和で一式そろう
Factory f = getFactory("japanese");
Plate p = f.createPlate();    // 和の器
Chopsticks c = f.createChopsticks(); // 和の箸
```

【つなぎり】 こうした「物を作る役」と並んで、処理の流れを受け持つ担当を分けたものが、次の「サービスクラス」です。

4. サービスクラス (Service Class)

ひとことと言うと:ある仕事のための処理(手続き)をひとまとめにして受け持つ、担当役のクラスのこと。

レストランの「予約受付係」のように、ひとつの役割をまるごと引き受ける担当のようなものです。予約に関わる確認・登録・通知をこの係に集めておくと、どこに何を頼めばよいか分かりやすくなります。

◆ サービスクラスがすること

- ひとつの仕事に必要な処理をまとめて持つ
- 画面やデータ保存の役と切り分けられる
- 同じ仕事を別の場所から呼び出して使える

【つなぎり】 こうした処理を「決めたタイミングで自動で動かす」ための仕組みが、次の「hooks」です。

第 2 部 — フックで自動化する(基礎)

5. hooks(フックス / Claude Code のフック機能)

ひとことと言うと:決めたタイミングで、あらかじめ用意した処理を自動で差し込める Claude Code の仕組みのこと。

人が近づくと自動で開く「自動ドアのセンサー」のようなものです。「この出来事が起きたら、この処理をする」とだけ決めておけば、あとは自分で見張らなくても自動で動いてくれます。

◆ hooks でできること(例)

- ファイルを書き換えたら自動で整形する
- コマンドを実行する前に内容を確認する
- 作業の節目に通知を送る

【つながり】 この仕組みの中で、「きっかけ」と「実行する処理」を 1 組にした設定そのものが、次の「フック」です。

6. フック(Hook / 1 つのフック設定)

ひとことと言うと:「このきっかけで、この処理を動かす」という結びつけを一つ定めた設定のこと。

「呼び鈴が鳴ったら玄関に出る」という一つの約束ごとのようなものです。きっかけ(呼び鈴)と、すること(玄関に出る)を一対一で結んだ、自動化の最小の一単位です。

◆ フック 1 つの形(イメージ)

```
// きっかけ: ファイルを書き換えたとき
// すること: 整形コマンドを実行する
{ "event": "PostToolUse", "command": "format.sh" }
```

【つながり】 そのフックが「どの相手のときに反応するか」を選ぶ条件が、次の「matcher」です。

7. matcher(マッチャー / 条件指定)

ひとことで言うと:フックを「どの対象のときに動かすか」を選び分けるための条件指定のこと。

人感センサーの「反応する範囲」を決めるダイヤルのようなものです。範囲を絞れば対象のときだけ反応し、広げればいろいろな相手に反応する、フックの当たり判定を決める設定です。

◆ matcher の指定(イメージ)

```
// ファイル書き込み系のツールのみで反応させる
"matcher": "Write|Edit"
// → Write か Edit のときに、このフックが動く
```

【つながり】 その「いつ動かすか」のうち、処理が実行される前を狙うきっかけが、次の「PreToolUse」です。

8. PreToolUse(プレ ツール ユース)

ひとことで言うと:ツールが実際に動く「前」に差し込まれる、フックのきっかけのこと。

ビルの入室ゲートで、中に入る前に行う「持ち物チェック」のようなものです。処理が走る前に内容が確かめられるので、危険なコマンドをここで止める、といった事前の関所に使えます。

◆ PreToolUse が向く使い方

- 実行してよい内容かを事前に確かめる
- 危険なコマンドをここで止める
- 実行前に必要な準備をしておく

【つながり】 前があれば後ろもあります。処理が終わった「後」を狙うきっかけが、次の「PostToolUse」です。

9. PostToolUse(ポスト ツール ユース)

ひとことで言うと:ツールが動き終わった「後」に差し込まれる、フックのきっかけのこと。

退室ゲートで、出るときに鳴る「お疲れさまの合図」のようなものです。処理が終わったあとに動くので、書き換えた直後に自動で整える、結果を通知する、といった後片づけに向きます。

◆ PostToolUse が向く使い方

- 書き換えた直後に自動で整形する
- 結果を記録に残したり通知する
- 後片づけのコマンドを走らせる

【つながり】では、そのきっかけで実際に「何のツールを動かすか」。代表格が、次の「Bash」です。

第3部 — フックの中身を書く

10. Write|Edit(ライト / エディット)

ひとことで言うと:ファイルの新規作成や書き換えを行うツール名で、`matcher` で狙うときの代表的な対象のこと。

ノートに「新しく書く(Write)」と「書き直す(Edit)」の二つの動作のようなものです。縦棒「|」は「または」の意味で、Write か Edit のどちらかのときに反応させたい、という指定になります。

◆ 「|」の読み方(イメージ)

```
"matcher": "Write|Edit"  
// | は「または」  
// → 新規作成(Write) または 書き換え(Edit) のとき
```

【つながり】 そのとき実際に走らせる処理を書く、定番の道具が、次の「Bash」です。

11. Bash(バッシュ)

ひとことで言うと:コマンドを打ち込んでコンピュータに指示を出す、定番の操作窓口(シェル)のこと。

お店のスタッフに「これをやって」と口頭で頼む受付窓口のようなものです。フックの中では、この窓口に向けて「整形して」「テストして」といった指示(コマンド)を渡して動かします。

◆ Bash で渡す指示(イメージ)

```
# ファイルを整形してからテストを走らせる  
npm run format  
npm test
```

【つながり】 その指示を一行ずつではなく、まとめて手順書にしておいたものが、次の「シェルスクリプト」です。

12. シェルスクリプト (Shell Script)

ひとことと言うと:実行したいコマンドを順番に書き並べて、まとめて動かせるようにした手順書のこと。

料理の手順を上から順に書いた「作業指示メモ」のようなものです。毎回コマンドを打ち直さなくても、このメモを実行すれば決まった手順を一気に通せます。

◆ シェルスクリプトの中身(イメージ)

```
#!/bin/bash
echo "整形します"
npm run format    # 上から順に実行される
npm test
```

【つながり】 そのスクリプトに、フック側から「今どんな状況か」を伝える渡し方が、次の「stdin 経由」です。

13. stdin 経由(標準入力ひょうじゅんにゅうりょく経由)

ひとことと言うと:プログラムの決まった「投入口」から、処理したいデータを流し込んで渡すやり方のこと。

受付の「投入口」に書類を入れて中の人に渡すようなものです。フックは「今どのファイルに触ったか」といった情報を、この投入口(標準入力)からスクリプトへ流し込みます。

◆ stdin で受け取る(イメージ)

```
# 投入口から来た内容を読み取って使う
data=$(cat)    # 標準入力をまるごと受け取る
echo "受け取った情報: $data"
```

【つながり】 流し込まれた情報を見て、最初に「対象かどうか」をざっと選り分ける関門が、次の「一次フィルタ」です。

14. 一次フィルタ (First-pass Filter)

ひとことと言うと:本格的な処理の前に、対象かどうかをざっと選り分ける最初の関門のこと。

郵便を仕分ける前に、宛先で大きく振り分ける「最初のふるい」のようなものです。関係ないものを入口で早めに弾いておくと、後の重い処理にかかる無駄が減ります。

◆ 一次フィルタの考え方

- 明らかに対象外のものを入口で除く
- 残ったものだけを次の処理へ回す
- 全体を軽く速く保てる

【つながり】 こうしたフックの考え方が、現場で最もよく使われる実例が、次の「pre-commit フック」です。

第 4 部 — フックの実例と仕上げ

15. pre-commit フック(プリコミット フック)

ひとことと言うと:変更を記録(コミット)する直前に、自動でチェックを走らせる仕掛けのこと。

提出前に行う「持ち物チェック」のようなものです。コミットという提出の直前に、整形や簡単な検査を自動でかけて、不備のあるものが混ざるのを未然に防ぎます。

◆ pre-commit でよくやること

- コードの整形やスペルチェック
- 簡単なテストや文法チェック
- 秘密情報の混入チェック

【つながり】 そのチェックの一つとして「ちゃんと組み上がるか」を確かめる号令が、次の「ビルドコマンド」です。

16. ビルドコマンド(Build Command)

ひとことと言うと:書いたコードから、実際に動く成果物へ組み上げる作業を起動する号令のこと。

部品を集めて完成品に「組み立てる」工場の号令のようなものです。この一声で、翻訳・部品の取りまとめ・梱包までが順に走り、動かせる形ができあがります。

◆ ビルドコマンドの例(イメージ)

```
npm run build    # JavaScript 系の組み立て
mvn package      # Java 系の組み立て
# → 動かせる成果物ができあがる
```

【つながり】 この「組み立て→検査→反映」を自動の流れ作業にまとめた仕組みが、次の「CI/CD パイプライン」です。

17. CI/CD パイプライン(シーアイ シーディー パイプライン)

ひとことと言うと:コードの検査から組み立て・反映までを、自動でつないで流す一本の流れ作業のこと。

工場の「ベルトコンベア」のようなものです。変更を載せると、検査の工程・組み立ての工程・お届けの工程を自動で順に通じ、人手の取りこぼしを減らします。

◆ パイプラインの流れ(イメージ)

工程	すること	たとえ
検査(CI)	テストや文法チェック	品質チェックの工程
組み立て	動く成果物にビルド	部品を完成品に組む工程
反映(CD)	本番などへ自動で届ける	出荷・お届けの工程

【つながり】 こうした流れの手前で、よく使う操作を一語で呼び出せるようにする近道が、次の「カスタムスラッシュコマンド」です。

18. カスタムスラッシュコマンド(Custom Slash Command)

ひとこと言うとは:よく使う一連の指示を、自分で名前を付けて「/名前」で呼び出せるようにした近道のこと。

電話機の「短縮ダイヤル」のようなものです。長い手順を毎回打ち込む代わりに、好きな名前を登録しておけば、その一語でまとめて呼び出せます。

◆ こんなときに便利

- 決まった作業を毎回同じ手順で呼び出す
- 長い指示を短い名前にまとめる
- チームで同じ手順を共有する

【つながり】 同じく端末から使う別システムの相棒ツールとして知られるのが、次の「Codex CLI」です。

19. Codex CLI(コーデックス シーエルアイ)

ひとことと言うと:端末から対話しながらコードを書かせられる、別システムの AI コーディング用コマンドツールのこと。

同じ厨房に立つ「もう一人の料理人」のようなものです。Claude Code と似た役割を端末で果たす相棒で、使い勝手や得意分野を見比べる対象としてよく挙げられます。

◆ 押さえておきたい点

- ターミナルから対話的に使う AI ツール
- Claude Code と立ち位置が近い
- 得意分野や操作感を比べて選べる

【つながり】 こうした手元のツールから一歩進み、外部のサービスとつなぐ共通の仕組みが、次の「MCP」です。

第 5 部 — 外部サービスとつなぐ(MCP)

20. MCP(エムシーピー / Model Context Protocol)

ひとことで言うと:AI と外部サービスを、共通の決まりごとでつなぐための連携の仕組みのこと。

いろいろな機器をつなげる「共通コンセント(変換ハブ)」のようなものです。サービスごとにバラバラなつなぎ方を覚えなくても、共通の差し込み口に合わせれば、AI から外部を扱えます。

◆ MCP でつなげる相手の例

- 課題管理(Jira など)
- データベース(PostgreSQL など)
- 連絡ツール(Slack など)

【つながり】 その共通の差し込み口で、多くの相手とのやり取りを一手に束ねる中心役が、次の「ハブ」です。

21. ハブ(Hub)

ひとことで言うと:多くの相手とのやり取りを、一箇所に集めて束ねる中心役のこと。

会社の「総合受付」のようなものです。あちこちに個別に行かなくても、まずここに来れば各所へ取り次いでくれる、連絡の集約点になります。

◆ ハブがあると

- つなぎ先が増えても窓口は一つで済む
- やり取りの流れを一箇所で見渡せる
- 個別のつなぎ込みの手間が減る

【つながり】 その中心役と個々のサービスとの間で、形の違いを吸収してつなぐ橋渡し役が、次の「ブリッジ」です。

22. ブリッジ(Bridge)

ひとことと言うと:仕組みの違う二つの間に立って、橋渡し役としてやり取りをつなぐ部品のこと。

言葉の違う二人の間に入る「通訳(橋)」のようなものです。片側の言い方をもう片側の分かる形に直して受け渡すので、直接つながらない二者でもやり取りできます。

◆ブリッジの役割

- 片側の形をもう片側の形に変換する
- 直接つながらない二者の間を取り持つ
- 片方を入れ替えても影響を小さくできる

【つながり】では、その橋の先にいる具体的な相手を見ていきます。まず課題を管理する付箋ボード役が、次の「Jira」です。

23. Jira(ジラ)

ひとことと言うと:やるべき課題やタスクを登録し、進み具合を管理する課題管理ツールのこと。

壁いっぱいの「付箋ボード」のようなものです。一枚ごとに「やること」を貼り、未着手・対応中・完了と動かしていくので、誰が何をどこまでやったか一目で分かります。

◆Jiraで見えること

- やるべきことの一覧と担当
- それぞれの進み具合(状態)
- 期限や優先度の整理

【つながり】その課題に紐づくデータを、大量に整理して保管しておく倉庫が、次の「PostgreSQL」です。

24. PostgreSQL(ポストグレ エスキューエル)

ひとことと言うと:大量のデータを表の形で整理して保存・検索できる、定番のデータベースのこと。

整理棚が並んだ「大型倉庫」のようなものです。データを決まった棚(表)にしまっておき、「この条件の物を出して」と頼めば必要なものだけ取り出せます。

◆ データの取り出し(イメージ)

```
-- 「対応中」の課題だけを倉庫から取り出す  
SELECT * FROM tasks  
WHERE status = 'in_progress';
```

【つながり】 そうして扱った情報を、関係する人へすぐ知らせる連絡掲示板が、次の「Slack」です。

25. Slack(スラック)

ひとことで言うと:チームの連絡を、話題ごとの部屋に分けてやり取りするビジネス向けチャットのこと。

話題ごとに分かれた「連絡掲示板」のようなものです。用件ごとに部屋(チャンネル)を分けて書き込むので、関係する人だけにまとまった形で連絡が届きます。

◆ Slack の使われ方

- 話題ごとのチャンネルで連絡する
- 自動の通知を流す先に使える
- やり取りが記録として残る

【つながり】 この連絡の仕組みを使い、勤怠の打刻もれなどを自動で知らせる実例が、次の「Timetracker-notification」です。

26. Timetracker-notification(タイムトラッカー ノーティフィケーション)

ひとことで言うと:勤怠の打刻もれなどを見つけて、対象者へ自動で知らせる通知の仕組みのこと。

「目覚まし時計のリマインド」のようなものです。打刻し忘れていないかを自動で見回り、抜けがあれば Slack など本人にそっと知らせて、後の困りごとを防ぎます。

◆ 通知の流れ(イメージ)

- 打刻データを定期的にかめる
- もれや異常を見つけたら抽出する
- 対象者へ自動でメッセージを送る

【つながり】 こうして外部サービスと自動でつながる仕組みは、第 1 部の「デザインパターン」や「サービスクラス」で整えた土台の上に成り立っています。

付録 A — 全体まとめ表

用語	ひとことと言うと	身近な例え
デザインパターン	定番の作り方の見本集	料理の定番レシピ集
ファクトリクラス	頼むと部品を作って渡す窓口	厨房の注文窓口
抽象ファクトリ	関連部品を一式まとめて作る	コースを丸ごとそろえる工場長
サービスクラス	ある仕事の処理を受け持つ担当	レストランの予約受付係
hooks	決めた時に処理を自動で差し込む	自動ドアのセンサー
フック	きっかけと処理を結んだ 1 設定	呼び鈴が鳴ったら玄関に出る約束
matcher	どの対象で動かすかの条件	人感センサーの反応範囲ダイヤル
PreToolUse	処理が動く前のきっかけ	入室前の持ち物チェック
PostToolUse	処理が終わった後のきっかけ	退室時のお疲れさまの合図
Write Edit	新規作成または書き換えの対象	ノートに書く・書き直す動作
Bash	コマンドで指示を出す操作窓口	口頭で頼む受付窓口
シェルスクリプト	コマンドを並べた手順書	上から順の作業指示メモ
stdin 経由	投入口からデータを流し込む	受付の投入口に書類を入れる
一次フィルタ	本処理の前にざっと選り分ける	郵便仕分け前の最初のふるい
pre-commit フック	記録の直前に自動チェック	提出前の持ち物チェック
ビルドコマンド	動く成果物へ組み上げる号令	完成品に組み立てる工場の

		号令
CI/CD パイプライン	検査～反映を自動でつなぐ流れ	工場のベルトコンベア
カスタムスラッシュコマンド	よく使う指示を一語で呼ぶ近道	電話機の短縮ダイヤル
Codex CLI	端末で使う別系統の AI ツール	同じ厨房のもう一人の料理人
MCP	AI と外部を共通の決まりでつなぐ	いろいろな機器の共通コンセント
ハブ	やり取りを束ねる中心役	会社の総合受付
ブリッジ	違う二つを橋渡しでつなぐ	言葉の違う二人の間の通訳
Jira	課題と進み具合を管理する	壁いっぱいの付箋ボード
PostgreSQL	大量データを表で保存・検索	整理棚が並んだ大型倉庫
Slack	話題ごとに分かれたチャット	話題別の連絡掲示板
Timetracker-notification	打刻もれを自動で知らせる	目覚まし時計のリマインド

おわりに — 次の一歩

知らない言葉も、身近な例えとセットで覚えると、ぐっと頭に入ります。分からなくなったら、何度でも読み返してください。

◆ こんな進め方がおすすめ

- 「ひとことで言うと」だけ先に通して読む
- 身近な例えとセットで思い出す
- つながりをつたって、関係する用語をまとめて覚える

—— あなたのペースで大丈夫。応援しています! ——